Energy & Buildings 237 (2021) 110742

Contents lists available at ScienceDirect

Energy & Buildings

journal homepage: www.elsevier.com/locate/enb

Aggregated short-term load forecasting for heterogeneous buildings using machine learning with peak estimation

Amine Bellahsen, Hanane Dagdougui *

[a] Department of Mathematics and Industrial Engineering, Polytechnique Montréal, Montréal, QC, Canada

ARTICLE INFO

Article history:
Received 14 July 2020
Revised 30 December 2020
Accepted 9 January 2021
Available online 1 February 2021

Keywords:
Short-term load forecasting
Sequence to sequence (Seq2Seq)
Recurrent neural network (RNN)
Long short-term memory (LSTM)
Random forest
K-nearest neighbor (KNN)
Mean absolute percentage error (MAPE)
Root mean squared error (RMSE)
Heterogeneous buildings
Peak prediction

ABSTRACT

System operations and planning are crucial aspects of power system management. They aim to maintain the equilibrium of electricity supply and demand while ensuring reliable and secure power system operation. Consumers have to pay more for electricity during periods of high peak demand in various sectors. If consumers have knowledge about expected peak load ahead of time, such extra charges could potentially be avoided. Accurate energy demand forecasting, and therefore expected peak load information, not only will help to provide a reliable supply of electricity, but also can be useful in reducing the cost of electricity at the consumer level. In this paper, we develop a comparative study for aggregated short-term load forecasting using different data strategies and compare two prediction levels: predicting the aggregated load using a district-level data set, and performing predictions on a lower level and then aggregating them at the district level. After finding the best forecasting model and strategy, these accurate predictions will help to predict the percentage of peak over a certain subscribed power in the entire district. The results showed that the mean absolute percentage error is between 1.67% and 4.80% depending on the machine learning algorithm and the prediction horizon used.

© 2021 Published by Elsevier B.V.

1. Introduction

In the smart grid paradigm, it becomes essential to support consumers/users in their demand-side management using different programs. Demand response (DR) programs implement price-based or incentive-based schemes to encourage changes in the end-uses of electricity [1]. From a grid perspective, the most important aspect of managing energy consumption is to reduce peak load when market costs are high or the reliability of the network is endangered. Such actions create benefits not only for the electricity provider, but also for consumers. To respond appropriately to these DR problems, demand aggregator companies are starting to offer consumers the possibility to participate in DR activities using their shiftable appliances and controllable loads [2], by reducing loads during specific events, shifting high energy demand to off-peak hours, managing controllable loads in real time in the market, or proposing financial rewards based upon the amount of flexible load provided.

In order to make DR programs operational, reliable load prediction tools and energy management systems must be implemented at the aggregator level. Consumers' DR, when accurately determined and controlled by aggregators, can play a greater role in the existing wholesale electricity market. It can support the distribution network operator in managing DR at a high level while exploiting the flexibility potential and helping consumers to benefit from rewards or lower energy bills. Load prediction at the aggregator level can better support the management of load flexibility and transmission system operators to balance the supply and demand of total energy during peak consumption periods; the aggregator can ask the contracted consumers to provide load flexibility.

Efficient load forecasting models are needed to better manage load in day-ahead and intra-day markets. Recently, with the rapid expansion of smart information and communication technologies [3], conventional power systems have shifted to smart grids as these smart infrastructures can generate very high volume and high speed rates, beyond the capabilities of traditional power systems. In order to take advantage of this massive amount of generated data, machine learning [4], which is one of the most active approaches in many areas of research, seems to be a key solution to increase prediction performance and better harness the variability existing in each consumption profile.

In order to ensure efficient electricity distribution for each user, short-term predictions are used to optimize the planning of dis-

* Corresponding author.
E-mail addresses: amine.bellahsen@polymtl.ca (A. Bellahsen), hanane.dagdougui@polymtl.ca (H. Dagdougui).

https://doi.org/10.1016/j.enbuild.2021.110742
0378-7788/© 2021 Published by Elsevier B.V.

Nomenclature

Nomenclature

Seq2Seq	Sequence To Sequence
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
KNN	K-Nearest Neighbor
MAPE	Mean Absolute Percentage Error
RMSE	Root Mean Squared Error
DR	Demand Response
CNN	Convolutional Neural Network
MLP	Multi-layer Perceptron
FFNN	Feed Forward Neural Network
GT	Ground Truth
HE	Human Expertise
ACF	Autocorrelation Function
SHAP	SHapley Additive exPlanation

tributed energy resources, including control and operation in microgrids. They also allow for better detection of anomalies in electricity usage patterns [5–9].

Power system operational planning becomes more difficult when dealing with a variety of buildings with different electricity consumption profiles. These differences create challenges in the process of constructing precise load prediction models for a district containing many types of buildings. Moreover, compared to research on load prediction at the level of the individual users, research at the level of the aggregator remains insufficient [10,11] despite its advancement with the evolution of smart grid infrastructures [3].

Many studies have been reported in the literature to solve the problem of short-term load prediction, especially with the advancement of machine learning [12–15]. In [13], a combination of two state-of-the-art time series prediction models is investigated to create a multi-bidirectional recurrent neural network. In [14], the authors presented a neural network approach for short-term residential load forecasting. A deep-learning forecasting framework based on long short-term memory (LSTM) was implemented to address variability problems regarding the consumption history of an individual household. The framework's performance was compared to that of multiple strategies like simple feed forward neural networks and K-nearest neighbors. In [15], the authors performed a novel forecasting method for predicting energy consumption called COSMOS, which combines multiple short-term load forecasting models using a stacking ensemble approach. They also demonstrated experimentally that COSMOS outperforms many recent machine learning approaches. Kim et al. [16] proposed a novel multi short-term load forecasting model based on a combination of recurrent neural networks (RNNs) and one-dimensional convolutional neural networks (1-D CNNs) with an inception module. To assess the performance of the proposed model, the authors performed a day-ahead load forecasting exercise for three industrial distribution complexes in South Korea. The forecasting performance of the proposed method was better than that of other methods, such as multi-layer perceptron (MLP), RNN, and 1-D CNN. In [17], a new technique based on machine learning decision tree approaches is presented to estimate the energy consumption of commercial buildings. Data from an energy consumption survey of five commercial buildings and weather data were provided to train the models. The results showed that the gradient boosting approach achieves good results. In [18], a study is presented on forecasting the energy consumption of buildings using artificial intelligence. The authors concluded that decision trees, support vector machines and neural networks are the most practical approaches to predict the energy consumption of buildings. Ensemble methods were also evaluated by combining two or more different learning approaches. The results showed that the ensemble methods outperformed the individual approaches.

Prior to modelling the time series approach, it is crucial to select the variables most correlated with the load history in order to make reliable predictions, from historical load variables to external variables such as weather-related ones like in [19–21]. For instance, in [19], the authors conducted experimental research to investigate the effect of adding temperature and relative humidity on electricity demand in North California and demonstrated a decrease in prediction errors. As mentioned earlier, with the advancement of smart meter infrastructures, it becomes interesting to study load forecasting at an aggregation level using data collected from various smart meters, as was done in [22,23,10,24]. For example, in [24], the authors proposed an LSTM based framework applied to real residential smart meter data, in order to forecast the load at the aggregator level first by using the predictions at the individual level and aggregating them, and second, by using the aggregated data to make predictions directly at the aggregator level. They concluded that the first approach is better. The ultimate goal being to use the aggregated predictions to forecast the percentage of peak time over a subscribed power in a day-ahead horizon, several studies have been conducted in [20,25]. In [20], the authors proposed a dynamical method to predict the daily peak load value, focusing primarily on the impact of different multi-source data, such as the natural and social factors like temperature and holidays that lead to load variations. The authors incorporated a framework as follows: for each forecast day, they used a random walk with restart algorithm to select similar days and build an adequate training set to be able to apply the Support Vector Regression method and forecast the daily peak. Their framework ended up outperforming other methods that focused on other feature/variable selection approaches.

Despite the importance of these efforts, there is still a lack of studies that compare time series statistical approaches to get the best performance, and leverage the district's data and each of its heterogeneous buildings' data sets for load and peak estimation.

In this paper, a load prediction framework is proposed that compares various machine learning approaches. We aim to address the gaps in the literature by looking at three main contributions: (1) use five major machine learning algorithms for time series forecasting to compare their performance using three prediction horizons: 15 min, 2 h and 24 h ahead; (2) compare aggregating the forecasts of each building to forecasting the aggregated load directly, while using data strategies to select the right historical features; and (3) use the best prediction model and strategy to determine their accuracy in terms of percentage of peak over the district subscribed power.

The remainder of this paper is organised as follows. Sections 2 and 3 review the top three approaches used for multi-step ahead prediction and describe the time series prediction algorithms used. Section 4 presents the case study that is analysed in this paper and a historical electricity use analysis at both the building and district level. An analysis was conducted to show the importance of select-

ing the right historical variables. Section 5 presents the machine learning and deep learning architectures as well as a framework that summarises the building of the predictive models. Key load forecasting and peak estimation results are presented in Section 6 along with details about the execution time. In this section, we also present an interpretation of the contribution of the input features used by our best model. Section 7 presents some of the limitations of this study. Section 8 concludes the paper with a discussion of future work.

2. Multi-step ahead time series prediction

For multi-step ahead prediction tasks, there are a variety of possible inference methods, three of which are most commonly used [26]:

- The **recursive method** uses one-step ahead prediction models. The output prediction at each time step T can be used as the model input for the next prediction at time step $T+1$. This approach works well but experiences error accumulation.
- The **direct method** uses independent models for each time step within the forecast horizon, i.e., T models are created to forecast outputs at time $t+1, t+2, .., t+T$. The predictions are constructed independently from one another.
- The **MIMO method** was developed to address the need to avoid single-output mapping, which disregards stochastic dependencies among future values and therefore alters the accuracy of predictions. This approach avoids the error accumulation problem and takes into account the dependence of the predictions through time.

In this paper, we focus on the MIMO method because it has been effectively applied to a variety of real-world multi-step ahead time series forecasting tasks [27–29] and seems better suited to load forecasting.

3. Methodology

This section describes the computational tools on which the proposed work is based. From classical machine learning, the concepts of random forest and K-nearest neighbors are briefly presented. From deep learning, principles like feed-forward neural network and recurrent neural network (RNN) and some of its variants-long short-term memory (LSTM) and sequence to sequence (Seq2Seq) LSTM-are detailed.

3.1. Random forest

Random forest [30] consists of a large number of individual decision trees that operate as an ensemble. A decision tree is built by splitting the source set of input variables, constituting the root node of the tree, into subsets that constitute the successor children. At each time step, the variable that is selected to split is chosen based on the variance reduction it brings to the data when split. This process is repeated on each derived subset in a recursive manner until a stopping condition is met.

Random forest uses the principle of bagging (bootstrapped aggregating), which is a prediction technique used to reduce the variance of the prediction model. Several bootstrapped samples from the original data are sampled and used to build trees. The distinctive feature of these trees is that at each time step, the variable chosen to split is taken from a subset of variables instead of the whole set of variables. These independent trees are used afterwards to average the prediction results.

Random forest is also used to select features by measuring the importance of a certain feature compared to others. Also, based on the decision tree schema, random forest gives results with clear interpretation.

3.2. K-Nearest Neighbor (KNN)

K-nearest neighbors [31] is a non-parametric method proposed for classification and regression. Since this paper pertains to a regression task, the target is predicted by local interpolation of the targets associated with the K-nearest neighbors in the training set. Thus, the KNN algorithm does not require training, but it is sensitive to the local structure of the data. In our case, an input is a vector in a multidimensional feature space with a continuous value as the target load consumption prediction.

A good K can be found by performing hyperparameter optimization, which is explained in Section 5.4.

3.3. Feed Forward Neural Network (FFNN)

Feed forward neural networks, also known as multi-layer perceptrons (MLPs) [4] are neural network-based methods used for universal function approximation. An MLP is composed of an input layer and an output layer, with one or more hidden layers in between (see Fig. 1). Some of the hyperparameters of the MLP are defined by the number of hidden layers $l = 1, \ldots, L$ and their sizes, which determine the depth of the network and its complexity in terms of neurons.

Given a single input vector $\{x_1, \ldots, x_t\}$, the multi-layer neural network outputs the predicted values at the time steps $\{y_{t+1}, \ldots, y_{t+T}\}$. Specifically, for a certain vector $x \in R^t$ fed to the input of the network, the MLP's computation can be expressed as:

$$a_l = W_l^T . h_{l-1} + b_l, \quad l \in \{1, \ldots, L\} \tag{1}$$

$$h_l = \phi_l(a_l) \tag{2}$$

where $h_0 = x \in R^t$ and the output $y_t = h_L \in R^T$. The layer l is defined, with its weight matrix $W_l \in R^{n_{l-1} \times n_l}$ and bias vector $b_l \in R^{n_l}$. ϕ_l can be any type of non-linearity function.

The dimension of output layer matches the forecast horizon $\{t+1, .., t+T\}$. The dimension of the input vector depends also on the exogenous variables, not only on the historical variables. More details about the input features are given in Section 4.3.

Considering a training set composed of input and output vectors $(x^{(i)}, y^{(i)})$ for $i \in \{1, 2, \ldots, m\}$, the learning process aims to find the best possible configuration of the parameters $\Theta = (W, b)$ that minimises a loss function evaluating the difference between the predicted values $\hat{y}^{(i)} = f(x^{(i)}, \Theta) \in R^T$ and the ground truth (GT) $y^{(i)} \in R^T$.

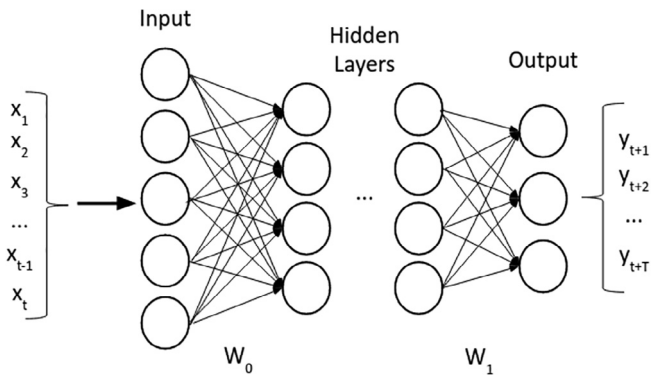


Fig. 1. Multi-layer perceptron.

Mean squared error is a very popular loss function that is used in regression problems and in time series in particular:

$$\mathscr{L}(\Theta) = \frac{1}{m}\sum_{i=1}^{m}||y^{(i)} - f(x^{(i)}, \Theta)||^2 \quad (3)$$

$$\hat{\Theta} \in \ \text{argmin}_{\Theta}\mathscr{L}(\Theta) \quad (4)$$

The Adam optimiser [32] is used to minimise the above loss function. It is one of the most efficient optimisation algorithms designed for training deep neural networks.

3.4. Recurrent neural networks and long short-term memory

Traditional feed forward neural networks do not necessarily learn the temporal aspect of the data because no sequential order is given to the inputs when they are fed to the neural network. In [4], the authors propose recurrent neural networks (RNNs) to overcome this drawback. RNNs are defined as networks that map the temporal relationship that exists in time series data using feed forward neural networks to capture the temporal component of the problem. In this case, the information relating to the state of the network in a time step is transferred to the next time step. This feature allows the persistence of information from an earlier time to teach upcoming new time steps.

3.4.1. RNN theoretical background

A simple recurrent neural network is also called a vanilla RNN or Elman network. The equations for a simple RNN are:

$$h_{t+1} = \sigma_h(W_x x_{t+1} + W_h h_t + b_h) \quad (5)$$

$$y_{t+1} = \sigma_y(W_y h_{t+1} + b_y) \quad (6)$$

where h_t is the hidden state at time step t, computed as a function of the input x_t and the hidden state from the previous time step h_{t-1}, using weight parameters W_x, W_h, W_y, bias parameters b_h, b_y, the hidden-layer activation function σ_h (generally $\sigma_h = tanh$) and the output activation function σ_y. y_{t+1} represents the target at time step $t+1$. Fig. 2 is a schematic representation of a vanilla RNN where 3 time steps are used as input (x_{t-1}, x_t, x_{t+1}) and 3 time steps are targeted $(y_{t+2}, y_{t+3}, y_{t+4})$.

The network in Fig. 2 immediately shows that if t represents the time, then the information is transferred with respect to each time step at a time, thus the temporal aspect is considered. This makes the RNN interesting for studying time series.

With this model, it should theoretically be possible to learn any relationship between the past and the current time even with long-term dependencies. However, sometimes in practice, RNNs fail to learn from these long-term dependencies. The problem, known as the vanishing gradient problem [33], is related to some interesting fundamental reasons why it might be difficult to learn from long-term dependencies. The long short-term memory (LSTM) neural network introduced by the authors in [34] is a successful approach to overcome this problem. LSTM has recently become more popular, and it works extremely well on a wide variety of problems, from time series forecasting [24] to image captioning [35].

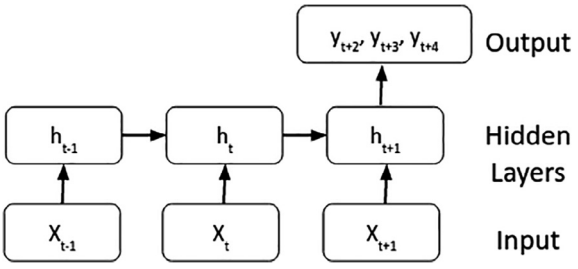


Fig. 2. Schematic of basic recurrent units.

3.4.2. LSTM

LSTM networks have a similar structure to the simple recurrent neural network. However, the main difference lies in the formation of the inner cell (see Fig. 3).

The distinctive feature of the LSTM architecture is that the inner architecture uses a gate system, which controls the processing of neural information. The aim of gated networks is to control the gradient stream. This feature helps to tackle the vanishing gradient problem. In detail, the neural calculation includes [36]:

$$f_t = \sigma(x_t.U^f + h_{t-1}.W^f) \quad (7)$$

$$i_t = \sigma(x_t.U^i + h_{t-1}.W^i) \quad (8)$$

$$o_t = \sigma(x_t.U^o + h_{t-1}.W^o) \quad (9)$$

$$\tilde{C}_t = tanh(x_t.U^g + h_{t-1}.W^g) \quad (10)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (11)$$

$$h_t = tanh(C_t) \odot o_t \quad (12)$$

where $W^{i,f,o,g} \in R^{p_H \times p_H}$ and $U^{i,f,o,g} \in R^{p_H \times p}$ are the weights that the network must adapt and learn, $\odot$ is the Hadamard product and σ is the sigmoid function.

The *gates* i_t, f_t, o_t are used to filter out information from the previous step, information from the current step and the final information. The purpose is to keep in memory only the important information based on the following explanations:

- f_t (*Forget gate*): It takes a value between 0 and 1. When it's close to 0, the effect of the old state is neglected according to Eq. (11).
- i_t (*Input gate*): It takes a value between 0 and 1. When it's close to 0, the effect of the current state is neglected according to Eq. (11).
- o_t (*Output gate*): It is used to filter the output of the network according to Eq. (12).

3.5. Seq2Seq LSTM

Sequence to sequence (Seq2Seq) architectures or encoder-decoder models [37] were originally designed to address the inability of RNNs to produce arbitrary-length output sequences. It was first used in neural machine translation, but it has proved to be the reference in many fields like image captioning and has recently been successfully used in time series forecasting [38].

The model consists of 2 parts, the encoder and the decoder:

- The encoder is a stack of recurrent units (RNN or LSTM cells for better performance), each of which is given an element of the input. It collects information about each element and passes it forward.
- The decoder is a stack of recurrent units, each of which predicts an element in the output sequence at a time step t, i.e, there will be T units when the predicted values are for $t+1, t+2, \ldots, t+T$.

Fig. 4 is a schematic representation of a Seq2Seq architecture, where 3 time steps are used as input (x_{t-1}, x_t, x_{t+1}) and 4 time steps are targeted $(y_{t+2}, y_{t+3}, y_{t+4}, y_{t+5})$.

Seq2Seq models provide insight that the temporal aspect needs to be considered, not only for the input, but also for the output. Although the encoder part is similar to the LSTM architecture, the decoder part treats the output in a novel way. In Fig. 4, y_{t+5} takes the information of the hidden layer at time $t+4$, which takes

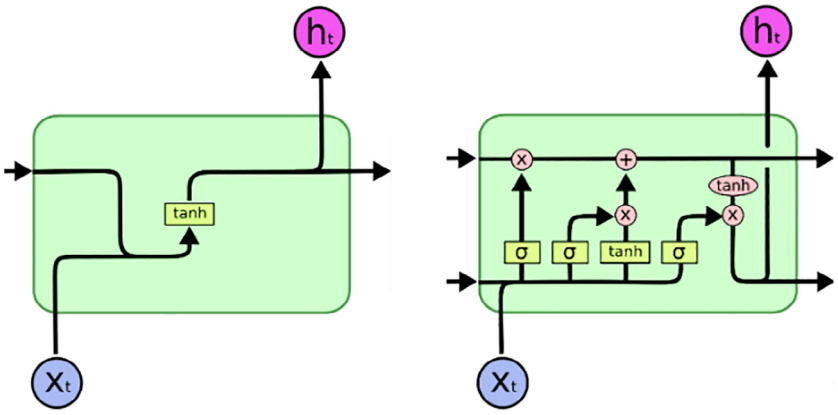


Fig. 3. Vanilla RNN - LSTM [36].

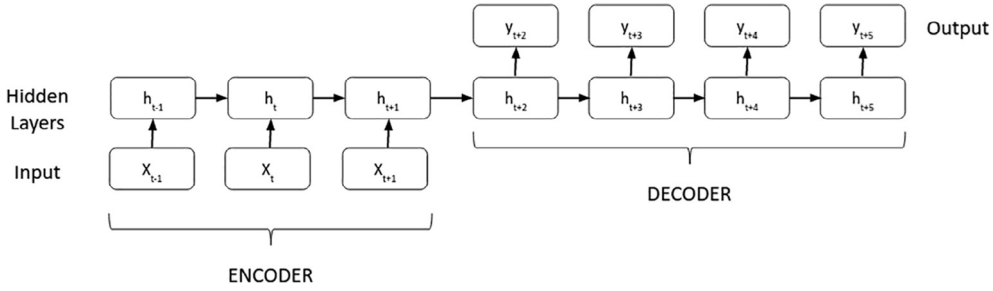


Fig. 4. Schematic of basic sequence to sequence architecture.

the information of all the hidden layers that come before it temporally (i.e., $t+3, t+2$).

4. Data analysis and processing

4.1. Time series construction

This study uses data collected from the building energy management system of an urban educational district in the Montréal region. The district consists of five buildings including residential, commercial, educational and institutional buildings, and therefore represents a heterogeneous mix of load patterns.

The data include instantaneous power consumption information that was collected every 15 min for the whole district as well as for each building in the district in 2015 and 2016. Fig. 5 shows a box plot of the average weekly power consumption for the 5 buildings.

Buildings 1 and 2 have an institutional load profile, building 3 houses a computing center, and buildings 4 and 5 have residential and commercial loads.

It can be observed that a variety of load profiles are present. For instance, the computing center maintains the same trend on weekends and weekdays. Moreover, consumption in buildings 1 and 2 varies greatly, particularly on weekdays. Significant variability is observed in several buildings, which makes load forecasting a complex task.

Since electricity consumption behaviour depends upon several factors, such as variations in temperature, season, relative humid-

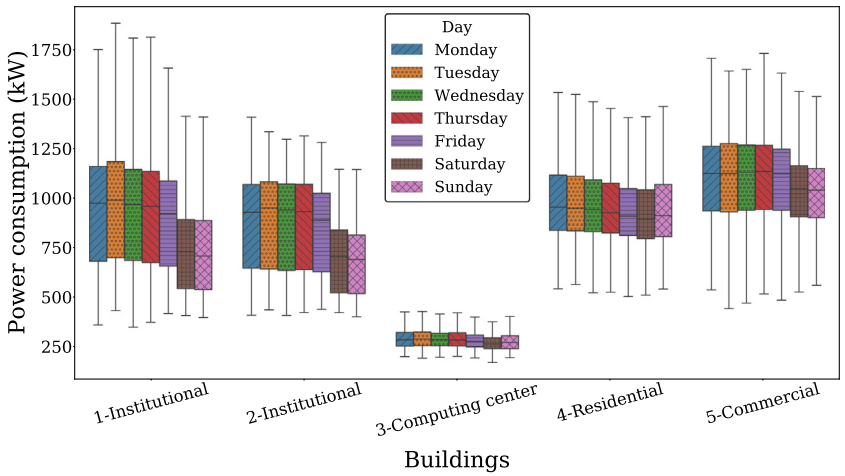


Fig. 5. Box plot of the average weekly power consumption of the 5 buildings.

ity, day of the week, and whether the day is a special holiday, we decided to add external features:

- Calendar data: week number, day number, hour of the day, quarter of the hour, working day/holiday.
- Meteorological data: temperature, dew point temperature, relative humidity

Adding these external features to the equations has provided great results and improved performance in several load forecasting studies [19].

4.2. Data normalisation

Many variables including power demand and temperature are very stochastic and follow a variety of value ranges and distributions. It therefore becomes necessary to apply a data normalisation strategy before constructing a load forecasting model. It is also useful to normalise the data to speed up the learning process and make the optimisation algorithm converge quickly. Normalisation was conducted using the *mean scale transformation* proposed in [39], which can be expressed as:

$$X_{new} = \frac{X}{1 + Mean(X)} \tag{13}$$

4.3. Historical inputs

In terms of time series modelling, it is crucial to select the features that will affect the prediction of electricity consumption in order to have the best possible performance. The data includes two types of variables:

- Historical variables: Those that contain the history of the load until the present time.
- External variables: Those that contain calendar and meteorological data.

Selecting historical variables often requires good knowledge of the data and the domain of the issue trying to be solved. That is why in many papers, historical variables are selected with the help of human expertise (HE) [23,24], where experts use their prior knowledge to select specific lags (time steps) of power history. Another way to select historical variables is to perform a technical analysis as presented in [40,29,41].

In this study, we compare performance following the use of human expertise with performance following the use of a strategy called the *autocorrelation analysis*.

4.3.1. Human Expertise (HE)

Predicting load consumption 1 h ahead might require, for example, prior knowledge of the load consumption for the last 4 h or week. Sometimes, it might also require knowing the load consumption 12 and 13 h ago. This selection can be problematic as is it important to find the right set of past historical lags to input in the prediction model to increase its performance. In many research papers, the choice of these historical lags relies on human expertise that uses prior knowledge in the domain to select the right lags (e.g., historical data for the last 2 h and week to predict future consumption). A human expertise approach is used in this paper, choosing the last 2 h and week as historical consumption lags in order to predict different horizons ahead.

4.3.2. Autocorrelation function

Along with the human expertise strategy, we also propose another, more quantitative, approach to historical variable selection using the *autocorrelation function* (ACF), which specifies the correlation between the observation at the current time step and the observations at previous time steps. Since each time step corresponds to 15 min, the goal here is to select the past time steps that are most correlated with time step t. Fig. 6 shows the relationship between time step t and 4 different previous time steps. The points cluster along a diagonal line from the bottom left to the top right of the plot. This demonstrates positive correlation. Moreover, a strong relationship is implied when the points are tighter to the diagonal line. The point cloud becomes sparser as the correlation value decreases when the lags get larger.

For more generalisation, an *autocorrelation plot* was made to help find the time steps most correlated with time step t. Fig. 7 represents this autocorrelation plot using the absolute values of these correlations. The t *vs* $t-7$ *days* point shown in red had a particularly strong relationship compared to most of the other time steps, which indicates that adding the power consumption of 7 days before to the historical inputs might be a good strategy.

The 15 best historical variables were selected based on their correlation with time $t+1$. When the district data is used, these variables include: $t, t-15$ *min*, $t-7$ *days*, $t-8$ *days*.

5. Architectures and framework

5.1. Data splitting

Data was split 60%/20%/20% into training, validation and testing sets with the rule as follows: the training set corresponds to data from 08/01/2015 to 08/03/2016, the validation set contains data from 08/03/2016 to 08/08/2016, and the testing set contains data from 08/08/2016 to 24/12/2016. The validation set was used to evaluate the trained model and help to determine which hyperparameters should ensure good test set performance.

5.2. Model regularisation

Deep learning techniques sometimes suffer from over-fitting, which results in the model fitting the training data perfectly and subsequently fitting the validation and test sets poorly. This study uses a regularisation method called *early stopping* [42], which is a generalisation method that quits the training process by controlling validation loss before reaching the maximum number of iterations. In this case, the training process terminates after a specified number of iterations when the error result in the validation set stops decreasing. This process is applied to the MLP, LSTM and Seq2Seq models.

5.3. Framework & performance metrics

In order to find the best strategy for short-term aggregated load prediction, we need to be able to answer the following questions:

- Which machine learning architecture model works the best?
- Which historical features selection method (HE or ACF) works the best?
- Is it better to have a single prediction model for the aggregated data, or have one model per building and then aggregate their predictions?

We use two metrics and a significance test to validate and measure the performance of our prediction models:

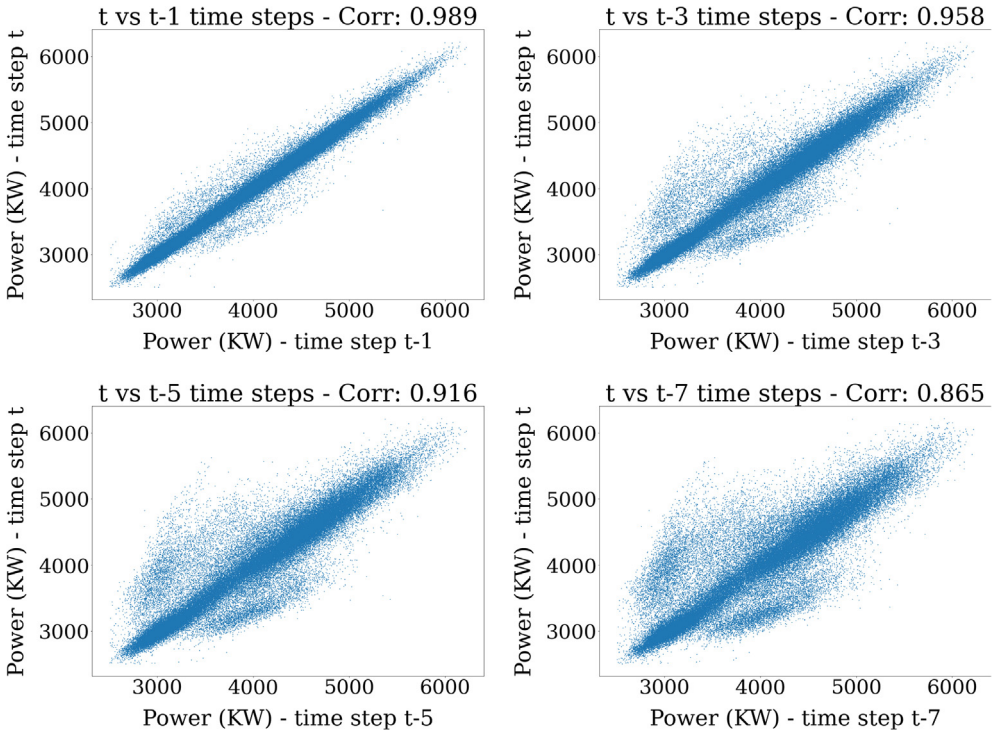


Fig. 6. Lag plots between time step t and 4 different previous time steps.

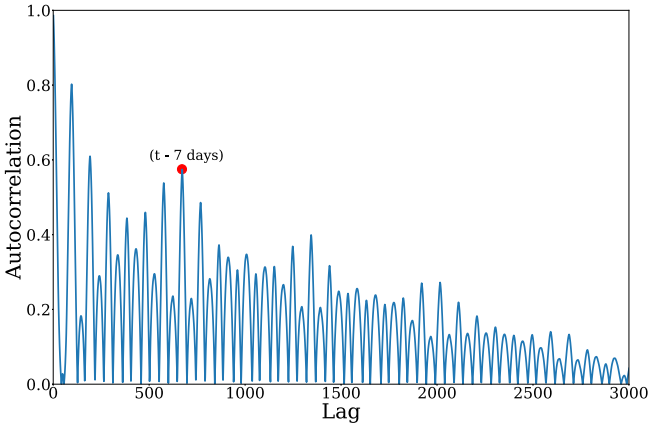


Fig. 7. Autocorrelation plot: autocorrelations for data values at varying time lags.

- The *mean absolute percentage error* (MAPE) metric:

$$MAPE = \frac{100\%}{n}\sum_{t=1}^{n}\left|\frac{G_t - P_t}{G_t}\right| \tag{14}$$

where G_t corresponds to the ground truth and P_t corresponds to the predicted value.

- The *root mean squared error* (RMSE) metric:

$$RMSE = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(G_t - P_t)^2} \tag{15}$$

where G_t corresponds to the ground truth and P_t corresponds to the predicted value.

- The *Wilcoxon signed-rank test*, also called the *Wilcoxon T test*, is a non-parametric statistical test used to determine if paired data samples were drawn from populations with the same or different distributions. The default assumption for the test, the null hypothesis (H_0), is that the two samples have the same distribution:
  - Fail to reject H_0: Sample distributions are equal.
  - Reject H_0: Sample distributions are not equal.

Since the focus is on short-term horizons, the proposed models are used to predict power consumption 15 min (i.e., 1 output value), 2 h (i.e., 8 output values) and 24 h (i.e., 96 output values) ahead. These predictions will be useful to have insight to measure the monthly percentage of peak over a subscribed power, in this case $5MW$.

5.4. Architecture selection

To find the right hyperparameters for each of the 3 models (MLP, LSTM, Seq2Seq), a random search, which is an iterative approach, was performed as follows:

- Randomly choose a combination of model hyperparameters:
  - For MLP, LSTM, Seq2Seq:
    - number of layers $\in \{2, 3, 4, 5\}$
    - number of neurons $\in \{32, 64, 128, 256\}$
  - For KNN:
    - Parameter K $\in \{5, 10, 20, 30, 50\}$
  - For random forest:
    - Number of trees $\in \{50, 80, 100, 120, 150\}$
- Train the model using the training set.
- Evaluate the model using the validation set, and save its performance.

The hyperparameter combination chosen will be the one that provides the best performance with the validation set.

For the random forest and KNN algorithms, we concluded that the best architectures are obtained for $K = 50$, followed by a ran-

dom forest with 100 independent trees. The other parameters were kept at the Scikit-learn [43] library default setting.

The MLP, LSTM and Seq2Seq models were each trained for 100 epochs using early stopping. After performing a hyperparameter search specifically on the number of neurons and hidden layers and the activation functions, we concluded that the best architectures are obtained for an MLP with 3 hidden layers (*layer 1*: 64 neurons, *layer 2*: 64 neurons, *layer 3*: 32 neurons), followed by an LSTM with 3 hidden layers (*layer 1*:64 neurons, *layer 2*: 128 neurons, *layer 3*: 32 neurons) and a Seq2Seq with 3 hidden layers for the encoder (*layer 1*: 64 neurons, *layer 2*: 64 neurons, *layer 3*: 32 neurons), and 1 hidden layer for the decoder (*layer 4*: 32 neurons). Other parameters such as the activation functions were kept at the Keras [44] library default setting.

5.5. Contribution of variables

Once the best model is found and its performance is measured, we might want to know more about the predictions and information about the contribution of the variables can help us better understand the data or even determine why our model fails if it does. Thus, it is important to understand the contribution of input variables. To do so, we use the SHapley Additive exPlanations (SHAP) approach [45], which is a method used to explain the predictions of a model by assuming that each variable is a player in a game where the prediction is the payout. SHAP values calculate variable importance by comparing what the prediction model predicts with and without the feature. That allows for a fair distribution of the payout among the variables used. This method is considered model-agnostic since it can be applied the same way to any machine learning model, which makes it more flexible than model-specific methods. Also, this method is one of the explanation methods that has a solid theoretical foundation in game theory.

6. Results and discussions

6.1. Execution time

The whole process from data acquisition to training/testing the models was performed on a computer with an Intel Core i7-6500U processor running at 2.5 GHz using 16 GB of RAM with NVIDIA GeForce 940 M, running on Ubuntu 18.04.

For the machine learning algorithms, random forest took 2h54 to train with all combinations of strategies for every prediction horizon, while the KNN did not take any time, since no training is required.

Due to the complexity of the deep learning models in terms of the number of layers and neurons, the MLP took 4h10 to train with all combinations of strategies for every horizon, while the LSTM took 25h50 and the Seq2Seq took 42h55. Also, the number of epochs for which each model was executed is not always the same due to early stopping.

In the testing phase, the execution time was nearly the same for the five models.

6.2. Power consumption forecasting

Tables 1–3 present the performance (MAPE (%) and RMSE (kW)) obtained using the test set with the proposed models. The MAPE results present the (mean $\pm$ std) for each strategy. The best performance for each model and prediction horizon is bolded.

Table 1 suggests that, apart from KNN, the four other algorithms have similar performance, whether using the district-level or building-level data set and using human expertise or the ACF. Even though the MLP performed second worst, we expect to have larger differences when predicting power consumption 2 h or 24 h ahead. This is attributed to the fact that predicting power consumption 15 min ahead requires a single output, while predicting 24 h ahead requires 96 outputs so the temporal aspect is more important in the 24-h case. The MLP does not necessarily take this temporal aspect into account. Also, the random forest algorithm performed the best when using the building-level data set and the ACF for the historical inputs. Table 2 illustrates the Wilcoxon signed-rank test results with respect to RMSE in terms of p_{value}, executed using the best models (in gray) and the 15 min ahead predictions. We compare the results of the best model (random forest) against the results of the other four models.

For a significance level $\alpha = 0.05$, the random forest results are significantly lower than those of the MLP or the KNN. However, they are statistically equal to those of the LSTM and Seq2Seq.

Table 3 indicates that KNN performed the worst and shows no difference in terms of performance compared to the 15 min ahead prediction. Again, the MLP performed second worst and is a bit further from the performance of the LSTM, Seq2Seq and random forest algorithms that have nearly the same MAPE & RMSE. Random forest performed the best using the building-level data set and the ACF for the historical inputs. Similarly to the 15-min ahead prediction, we ran a Wilcoxon signed-rank test to compare the results of the different models. However, this time, random forest proved to significantly outperform ($p_{value} \approx 0$) the other 4 models.

It can be seen in Table 4 that the MLP's MAPE and RMSE are increasing quickly compared to those of the other four algorithms, while the KNN's best performance appears to not change much from the previous predictions. This demonstrates that the temporal aspect of the problem plays an important role in building precise models. The LSTM and Seq2Seq algorithms again perform nearly the same, and random forest performs the best using the building-level data set and the ACF for the historical inputs. Similarly, the Wilcoxon signed-rank test shows that random forest significantly outperforms ($p_{value} \approx 0$) the other 4 models.

To conclude, the best overall model and strategy are:

- Using random forest as the prediction model.
- Predicting for each building and then aggregating the predictions.
- Selecting historical data for each building using ACF.

Although the recurrent neural networks performed similarly well as random forest, there are three main advantages to using random forest over any neural network model. First, random forest combines the predictions of multiple decision trees by aggregating their results. In other words, the approach is less prone to overfitting. Second, random forest is less computationally expensive as mentioned in Section 6.1. Last, random forest is a great tool to use when an understanding of the variables is required, since it gives an interpretation of the results, as it provides the contribution of each variable.

Fig. 8 shows the aggregated prediction versus the ground truth between Monday, Aug 8, and Sunday, Aug 16, 2016, using the best random forest model for the three prediction horizons. As expected, less precision is obtained when using more time steps (i.e., 24 h ahead).

The advantage of using the building-level data set is that since it requires training separate models for each of the five buildings, the ACF makes it possible to select the best historical inputs for each of the five buildings because it captures the internal behaviour of each one, which adds flexibility to our models. Moreover, the performance of each of the five models is measured, thus it gives an interpretation to what most affects performance for the sum of the predictions.

Table 1
Performance of 15 min ahead load prediction.

		District-level data set		Building-level data set	
		HE	ACF	HE	ACF
MLP	MAPE (%)	2.09 ± 2.41	2.00 ± 1.86	2.26 ± 2.65	**1.91 ± 2.15**
	RMSE (kW)	79.85	78.46	87.17	**73.74**
LSTM	MAPE (%)	2.08 ± 2.02	2.12 ± 2.10	2.04 ± 2.34	**1.96 ± 2.23**
	RMSE (kW)	79.61	81.34	78.46	**74.22**
Seq2Seq	MAPE (%)	1.88 ± 1.91	2.00 ± 2.02	2.09 ± 2.37	**1.83 ± 2.15**
	RMSE (kW)	72.94	76.28	79.67	**72.89**
Random Forest	MAPE (%)	1.76 ± 1.80	1.68 ± 1.60	1.79 ± 1.83	**1.67 ± 1.58**
	RMSE (kW)	68.36	65.56	69.10	**64.88**
KNN	MAPE (%)	7.80 ± 6.66	7.54 ± 6.35	7.33 ± 5.99	**6.61 ± 5.21**
	RMSE (kW)	307.76	298.16	288.24	**260.24**

Table 2
15 min ahead, Wilcoxon signed-rank test

	MLP	LSTM	Seq2Seq	KNN
Random Forest	$p \approx 0$	$p = 0.080$	$p = 0.697$	$p \approx 0$

Table 3
Performance of 2 h ahead load prediction.

		District-level data set		Building-level data set	
		HE	ACF	HE	ACF
MLP	MAPE (%)	4.47 ± 4.28	3.61 ± 3.52	4.70 ± 4.83	**3.26 ± 3.51**
	RMSE (kW)	172.16	137.30	171.69	**124.22**
LSTM	MAPE (%)	3.24 ± 3.16	3.35 ± 3.37	3.28 ± 3.04	**2.93 ± 2.95**
	RMSE (kW)	124.74	129.36	124.84	**113.64**
Seq2Seq	MAPE (%)	**2.93 ± 2.92**	3.66 ± 3.48	3.06 ± 3.09	3.33 ± 3.27
	RMSE (kW)	**113.68**	139.95	118.66	127.42
Random Forest	MAPE (%)	3.08 ± 3.43	2.88 ± 3.13	3.27 ± 3.40	**2.83 ± 2.73**
	RMSE (kW)	118.14	111.95	123.06	**109.04**
KNN	MAPE (%)	8.23 ± 6.83	7.70 ± 6.24	7.94 ± 6.47	**6.96 ± 5.34**
	RMSE (kW)	325.03	306.98	312.44	**275.18**

Table 4
Performance of 24 h ahead load prediction.

		District-level data set		Building-level data set	
		HE	ACF	HE	ACF
MLP	MAPE (%)	11.02 ± 9.75	**9.85 ± 7.82**	14.48 ± 11.60	11.24 ± 9.21
	RMSE (kW)	414.39	**398.93**	544.98	468.95
LSTM	MAPE (%)	5.99 ± 4.93	6.10 ± 5.24	5.86 ± 4.96	**5.21 ± 4.59**
	RMSE (kW)	234.96	242.42	233.24	**210.29**
Seq2Seq	MAPE (%)	6.14 ± 4.92	5.87 ± 5.15	**5.43 ± 4.85**	5.68 ± 4.97
	RMSE (kW)	240.49	235.27	**217.74**	225.16
Random Forest	MAPE (%)	6.63 ± 8.20	6.71 ± 7.67	7.54 ± 8.06	**4.80 ± 4.24**
	RMSE (kW)	252.30	256.20	281.35	**186.74**
KNN	MAPE (%)	11.21 ± 10.06	10.81 ± 9.73	11.16 ± 9.96	**6.80 ± 5.44**
	RMSE (kW)	426.03	412.73	422.28	**264.12**

This study confirmed that the models used to predict the power demand of buildings 1, 2 and 4 performed worse in terms of MAPE, as shown in Table 5.

This can be explained by the fact that some buildings are affected by more stochasticity related to their dependence on many parameters such as occupancy and the behaviour and habits of each occupant.

Load forecasting was performed for each prediction horizon using the best random forest model and strategy combination for 2015–2016. The box plot representation is depicted in Fig. 9, with MAPE (%) calculated daily then aggregated monthly.

The results obtained are reliable, especially for the 15 min and 2 h prediction horizons, for which MAPE variability is small compared to that of 24 h. It can be observed that May suffers the most from poor model performance, regardless of the prediction horizon. These values are affected by variation related to sociological activities and holidays, which directly affects power consumption during this period.

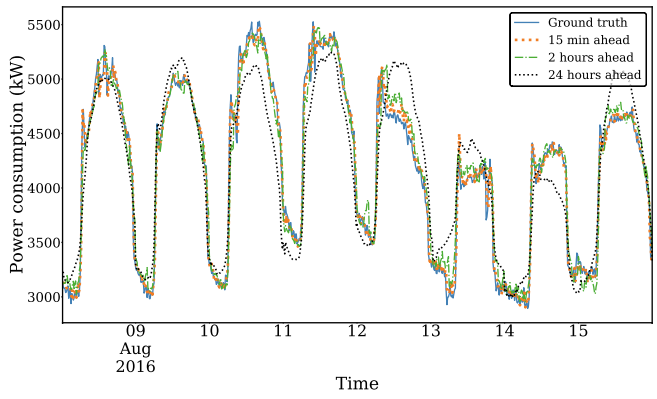


Fig. 8. Prediction vs Ground truth using random forest, the building-level data set and ACF for the 15 min/2 h/24 h prediction horizons.

Now that the best model has been selected, we apply the SHAP method to determine the contribution of each feature in the model. Fig. 10 represents the SHAP feature importance using the best prediction model and the 2 h ahead prediction horizon. The variable $T-i$ refers to power consumption at time $T-i$ where i is the number of quarter-hour units (e.g., $T-96$ refers to $T-24$ hours).

It is clear that the variables contributing the most are the historical ones. In fact, the most important ones are the lag directly before (i.e., T) and the lags around a day before ($T-92$) and a week before ($T-671$), which is compatible with previous intuitions and scientific knowledge. The plot also shows that other lags (e.g. $T-2$) don't contribute much to the predictions. It is important then to accurately use the historical data from 1 day and 1 week before. The meteorological features, such as external temperature and relative humidity, are also important to the predictions, since Canada experiences high temperatures during summer and very low temperatures during winter, which greatly affects power consumption related to heating and cooling. Finally, apart from "Hour of the Day", the calendar variables contributed the least, which might seem counterintuitive since the time of the year is important to infer power consumption. However, it is crucial to understand that the SHAP method allows us to explain the model rather than the data, thus it shows that the decision-making process of the random forest model relies heavily on historical variables and less on calendar data. Furthermore, due to the collinearity that exists between calendar data and meteorological data as well as historical data, this introduces a correlation bias to the training of our prediction models, which causes the models to heavily weight some representative features (e.g., historical ones) and neglect the effect of others (e.g., calendar data).

6.3. Estimation of the monthly percentage of peak over a subscribed power

In addition to performing well, the random forest model can also be used to estimate and extract useful information related to monthly peak load. We decided to study prediction model performance to accurately predict the percentage of peak times that are over the subscribed power. For this, we used the forecasts made by the best random forest model and strategy, and calculated the average number of times the prediction values were greater than the subscribed power. Finally, we compared it to the average number of times the ground truth values were greater than the same subscribed power in Fig. 11.

Fig. 11 shows that for the three forecasting horizons, the percentage of peaks that are over the subscribed power are similar to the ground truth results. Since the model's performance is accurate as indicated in Section 6.2 and the percentage of times when power predictions exceeded the subscribed power is similar to the percentage of times when power ground truth exceeded it, it can be concluded that these results are reliable in most cases.

7. Limitations

One of the main challenges of machine learning algorithms is the computational capacities they require, as mentioned in Section 6.1. In fact, since each training process depends significantly on the data set, the selected historical features and the different hyperparameters to use, a considerable range of combinations should be tested to compare performance. For this main reason, the hyperparameters search (number of layers, number of neurons, number of trees) seen in Section 5.4 is limited and not all combinations could be tested.

Another limitation concerns the poor performance obtained for the 24-h predictions in May in Section 6.2. This limitation could be addressed by adding an adaptation weight when training the model for May if deep learning models are used, or using boosting techniques if tree based learning is considered.

Furthermore, despite the solid performance of our final model, we strongly believe that the data would be much more usable to improve the model in question if we had access to consumption at the consumer level instead of the building level. Those data would serve to better capture the various behaviors at a lower level and thus make the prediction model more robust to abrupt changes in a consumer's consumption.

8. Conclusion and future work

This work compares the accuracy of various machine learning approaches for aggregated multi-step ahead predictions of electrical load using heterogeneous data. The forecasts were used to determine their reliability in terms of the percentage of peak over a certain subscribed power. Time series approaches were developed for district-level data and for building-level data. The simulation results show that jointly using the random forest algorithm and ACF with building-level data, then aggregating the results to predict the district load produces more accurate forecasts.

In regards of the interpretation of the predictions of aggregated power consumption, the results show that the historical features significantly impact the prediction, as they are very important to infer future power consumption. This aligns with the intuition and scientific knowledge we used to select the historical lags used in the input. Similarly, meteorological features such as external temperature and relative humidity also have a significant impact on the prediction value. Finally, apart from the "Hour of the Day" variable, calendar data did not significantly affect the random forest model, and that could be explained by the correlation bias that is introduced by the collinearity that exists between some of the input variables.

Table 5
Performance of each building for 24 h ahead prediction using random forest and ACF.

Building 1	Building 2	Building 3	Building 4	Building 5
7.348 %	5.196 %	3.564 %	7.015 %	6.127 %

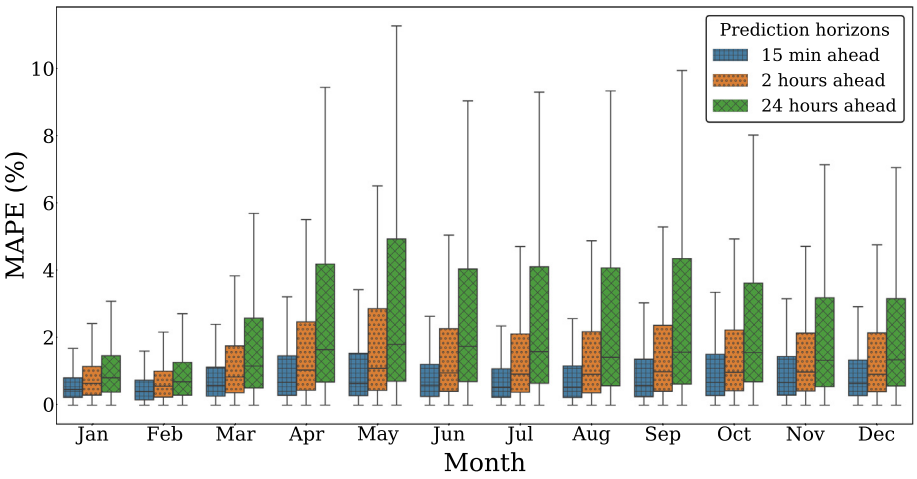


Fig. 9. Monthly MAPE distribution for each of the 3 prediction horizons.

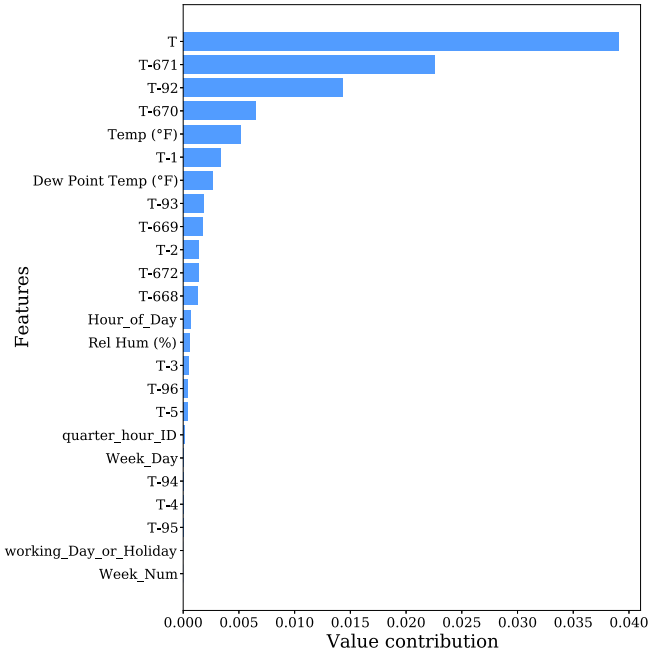


Fig. 10. Bar plot of feature importance – random forest.

Future work will explore various demand responses in order to support consumers in reducing their power demand, whether by price-based or incentive-based demand response. We will also investigate reinforcement learning strategies to perform peak shaving at the district level depending on each building's behaviour.

CRediT authorship contribution statement

Amine Bellahsen: Software, Methodology, Visualization, Writing - original draft, Writing - review & editing. **Hanane Dagdougui:** Conceptualization, Funding acquisition, Methodology, Supervision, Visualization, Writing - original draft, Writing - review & editing.

Declaration of Competing Interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: This research was supported by the NSERC-Hydro-Quebec-Schneider Electric Industrial Research Chair on Optimization for Smart grids.

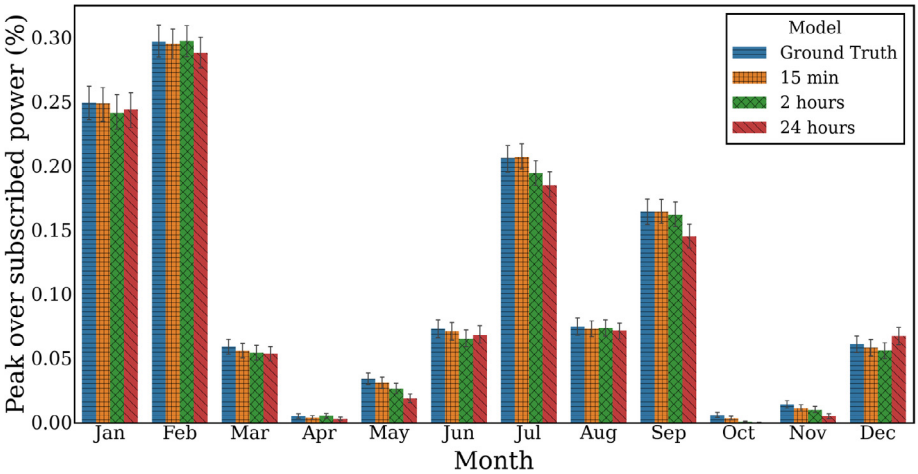


Fig. 11. Monthly percentage of peak over 5000 kW - ground truth vs predictions.

Acknowledgements

The authors would like to thank ÉTS Montréal for providing the data used for this work. This research was supported by the NSERC/ Hydro-Québec/Schneider Electric Industrial Research Chair on Optimization for the Smart Grid.

References

[1] R. Lu, S.H. Hong, Incentive-based demand response for smart grid with reinforcement learning and deep neural network, Applied Energy 236 (2019) 937–949.
[2] A.M. Carreiro, H.M. Jorge, C.H. Antunes, Energy management systems aggregators: A literature survey, Renewable and Sustainable Energy Reviews 73 (2017) 1160–1172.
[3] B.P. Hayes, J.K. Gruber, M. Prodanovic, Multi-nodal short-term energy forecasting using smart meter data, IET Generation, Transmission & Distribution 12 (2018) 2988–2994.
[4] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press (2016).
[5] A. Dedinec, S. Filiposka, A. Dedinec, L. Kocarev, Deep belief network based electricity load forecasting: An analysis of macedonian case, Energy 115 (2016) 1688–1700.
[6] A. Ouammi, H. Dagdougui, R. Sacile, Optimal control of power flows and energy local storages in a network of microgrids modeled as a system of systems, IEEE Transactions on Control Systems Technology 23 (2014) 128–138.
[7] C. Bersani, H. Dagdougui, A. Ouammi, R. Sacile, Distributed robust control of the power flows in a team of cooperating microgrids, IEEE Transactions on Control Systems Technology 25 (2016) 1473–1479.
[8] H. Dagdougui, A. Ouammi, R. Sacile, Optimal control of a network of power microgrids using the pontryagin's minimum principle, IEEE Transactions on Control Systems Technology 22 (2014) 1942–1948.
[9] H. Dagdougui, R. Sacile, Decentralized control of the power flows in a network of smart microgrids modeled as a team of cooperative agents, IEEE Transactions on Control Systems Technology 22 (2013) 510–519.
[10] H. Chen, S. Wang, S. Wang, Y. Li, Day-ahead aggregated load forecasting based on two-terminal sparse coding and deep neural network fusion, Electric Power Systems Research 177 (2019) 105987.
[11] S. Sreekumar, K.C. Sharma, R. Bhakar, Gumbel copula based aggregated net load forecasting for modern power systems, IET Generation, Transmission & Distribution 12 (2018) 4348–4358.
[12] M. Tan, S. Yuan, S. Li, Y. Su, H. Li, F. He, Ultra-short-term industrial power demand forecasting using lstm based hybrid ensemble learning, IEEE Transactions on Power Systems (2019).
[13] X. Tang, Y. Dai, T. Wang, Y. Chen, Short-term power load forecasting based on multi-layer bidirectional recurrent neural network, IET Generation, Transmission & Distribution 13 (2019) 3847–3854.
[14] W. Kong, Z.Y. Dong, D.J. Hill, F. Luo, Y. Xu, Short-term residential load forecasting based on resident behaviour learning, IEEE Transactions on Power Systems 33 (2017) 1087–1088.
[15] J. Moon, S. Jung, J. Rew, S. Rho, E. Hwang, Combination of short-term load forecasting models based on a stacking ensemble approach, Energy and Buildings 109921 (2020).
[16] J. Kim, J. Moon, E. Hwang, P. Kang, Recurrent inception convolution neural network for multi short-term load forecasting, Energy and Buildings 194 (2019) 328–341.
[17] C. Robinsona, B. Dilkinaa, J. Hubbse, W. Zhangb, S. Guhathakurtab, M.A. Brownc, R.M. Pendyalad, Machine learning approaches for estimating commercial building energy consumption, Applied Energy (2017) 889–904.
[18] Z. Wang, R.S. Srinivasan, A review of artificial intelligence based building energy use prediction: Contrasting the capabilities of single and ensemble prediction models, Renewable and Sustainable Energy Reviews (2017) 796–808.
[19] J. Xie, Y. Chen, T. Hong, T.D. Laing, Relative humidity for load forecasting models, IEEE Transactions on Smart Grid 9 (2016) 191–198.
[20] P. Zeng, M. Jin, Peak load forecasting based on multi-source data and day-to-day topological network, IET Generation, Transmission & Distribution 12 (2017) 1374–1381.
[21] Y. Wang, Q. Chen, N. Zhang, Y. Wang, Conditional residual modeling for probabilistic load forecasting, IEEE Transactions on Power Systems 33 (2018) 7327–7330.
[22] Y. Wang, Q. Chen, M. Sun, C. Kang, Q. Xia, An ensemble forecasting method for the aggregated load with subprofiles, IEEE Transactions on Smart Grid 9 (2018) 3906–3908.
[23] J. Ponoćko, J.V. Milanović, Forecasting demand flexibility of aggregated residential load using smart meter data, IEEE Transactions on Power Systems 33 (2018) 5446–5455.
[24] W. Kong, Z.Y. Dong, Y. Jia, D.J. Hill, Y. Xu, Y. Zhang, Short-term residential load forecasting based on lstm recurrent neural network, IEEE Transactions on Smart Grid 10 (2017) 841–851.
[25] Z. Yu, Z. Niu, W. Tang, Q. Wu, Deep learning for daily peak load forecasting–a novel gated recurrent neural network combining dynamic time warping, IEEE Access 7 (2019) 17184–17194.
[26] S.B. Taieb, G. Bontempi, A.F. Atiya, A. Sorjamaa, A review and comparison of strategies for multi-step ahead time series forecasting based on the nn5 forecasting competition, Expert Systems with Applications 39 (2012) 7067–7083.
[27] A. Motamedi, H. Zareipour, W.D. Rosehart, Electricity price and demand forecasting in smart grids, IEEE Transactions on Smart Grid 3 (2012) 664–674.
[28] M. Askari, F. Keynia, Mid-term electricity load forecasting by a new composite method based on optimal learning mlp algorithm, IET Generation, Transmission & Distribution 14 (2020) 845–852.
[29] C. Fan, J. Wang, W. Gang, S. Li, Assessment of deep recurrent neural network-based strategies for short-term building energy predictions, Applied Energy 236 (2019) 700–710.
[30] T.K. Ho, Random decision forests, in: Proceedings of 3rd international conference on document analysis and recognition, vol. 1, IEEE, pp. 278–282..
[31] T. Cover, P. Hart, Nearest neighbor pattern classification, IEEE Transactions on Information Theory 13 (1967) 21–27.
[32] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, arXiv preprint arXiv:1412.6980 (2014)..
[33] R. Pascanu, T. Mikolov, Y. Bengio, On the difficulty of training recurrent neural networks, in: International conference on machine learning, pp. 1310–1318..
[34] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Computation 9 (1997) 1735–1780.
[35] O. Vinyals, A. Toshev, S. Bengio, D. Erhan, Show and tell: A neural image caption generator, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 3156–3164..
[36] C. Olah, Understanding lstm networks (2015).
[37] I. Sutskever, O. Vinyals, Q.V. Le, Sequence to sequence learning with neural networks, in: Advances in Neural Information Processing Systems, pp. 3104–3112..
[38] A. Gasparin, S. Lukovic, C. Alippi, Deep learning for time series forecasting: The electric load case, 2019, arXiv preprint arXiv:1907.09207.
[39] K. Bandara, P. Shi, C. Bergmeir, H. Hewamalage, Q. Tran, B. Seaman, Sales demand forecast in e-commerce using a long short-term memory neural network methodology, arXiv preprint arXiv:1901.04028 (2019)..
[40] C. Heghedus, A. Chakravorty, C. Rong, Energy load forecasting using deep learning, in: 2018 IEEE International Conference on Energy Internet (ICEI), IEEE, pp. 146–151..
[41] M.A. Rahman, A. Zubair, et al., Electric load forecasting with hourly precision using long short-term memory networks, in: 2019 International Conference on Electrical, Computer and Communication Engineering (ECCE), IEEE, pp. 1–6..
[42] L. Prechelt, Early stopping-but when?, in: Neural Networks: Tricks of the Trade, Springer, 1998, pp. 55–69.
[43] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine learning in Python, Journal of Machine Learning Research 12 (2011) 2825–2830.
[44] F. Chollet, et al., Keras, https://keras.io, 2015..
[45] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, in: Advances in Neural Information Processing Systems, pp. 4765–4774..